SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

ERROR ANALYSIS TASK

Name of the Student	SHAM S.
Reg. No	192325076

COURSE NAME: Deep Learning **COURSE CODE: MLA0403**
FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality.
(BTL-4)

CO1 Weightage: 15%

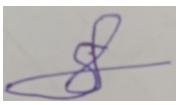
Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I SHAM S. (192325076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

QUESTIONS: **Total Marks:25**

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.
Tasks:

a) Identify the conceptual error in the likelihood formulation.

Answer:

The conceptual error is that the likelihood function is written as the **sum of probabilities** instead of the **product of probabilities**.

In Maximum Likelihood Estimation (MLE), the observations are assumed to be **independent**, so the likelihood is the product of the probability of each observation given the model parameters. Adding probabilities does not represent the joint probability of all observations.

---

b) Explain how this mistake affects parameter estimation.

Answer:

Using the sum of probabilities instead of the product leads to incorrect parameter estimation because:

- It does not represent the true probability of observing the entire dataset.
- The optimization objective becomes mathematically incorrect.
- The estimated parameters no longer maximize the actual likelihood.
- Even with a large dataset, the model converges to incorrect parameter values.
- This results in poor prediction accuracy and reduced classifier performance.

---

c) Suggest the correct mathematical formulation and reasoning.

Answer:

For independent observations $x_1, x_2, \ldots, x_n$, the correct likelihood function is:

$$L(\theta) = \prod_{i=1}^{n} P\left(x_i \mid \theta\right)$$

where:

- $L(\theta)$= likelihood function
- θ= model parameters
- $P(x_i \mid \theta)$= probability of observation x_i

Reasoning:

The product is used because the joint probability of independent observations equals the product of their individual probabilities. MLE chooses the parameter θthat maximizes this likelihood.

---

d) How would using log-likelihood fix computational issues?

Answer:

The log-likelihood is obtained by taking the logarithm of the likelihood:

$$\log L(\theta) = \log\left(\prod_{i=1}^{n} P(x_i \mid \theta)\right) = \sum_{i=1}^{n} \log P(x_i \mid \theta)$$

Using log-likelihood helps because:

- It converts multiplication into addition, making calculations simpler.
- It prevents numerical underflow caused by multiplying many very small probabilities.
- It improves computational efficiency.
- It makes differentiation and optimization easier for algorithms such as Gradient Descent and Newton's Method.
- Since the logarithm is a monotonic function, maximizing the log-likelihood gives the same optimal parameters as maximizing the likelihood.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

a) List possible implementation errors in the learning algorithm.

Answer:

Possible implementation errors include:

- Incorrect weight update formula.
- Bias term is not included or not updated.
- Wrong prediction function (incorrect threshold/sign function).
- Learning rate is set incorrectly.
- Labels are not encoded correctly (should typically be **+1** and **−1**).
- Weights are not initialized properly.
- Training data is not shuffled between epochs.
- Stopping condition or convergence check is implemented incorrectly.

---

b) Analyze how incorrect learning rate or weight updates affect convergence.

Answer:

- A **very large learning rate** causes weight updates to overshoot the optimal boundary, making the algorithm oscillate and fail to converge.
- A **very small learning rate** results in tiny weight changes, causing very slow convergence.
- An **incorrect weight update equation** updates the weights in the wrong direction, preventing the model from finding the correct decision boundary.
- Failure to update the **bias** correctly also prevents the perceptron from learning the appropriate separating hyperplane.

---

c) Identify dataset-related issues that may cause this problem.

Answer:

Possible dataset-related issues include:

- Incorrect class labels (e.g., using **0 and 1** instead of **−1 and +1** when the implementation expects the latter).
- Incorrect or noisy data labels.
- Missing or duplicate samples.
- Features with very different scales, making learning unstable.
- Data preprocessing errors such as missing values or incorrect normalization.
- Although the dataset is assumed to be linearly separable, implementation mistakes during data preparation may make it appear non-separable.

---

d) Propose modifications to ensure convergence.

Answer:

To ensure convergence:

- Use the correct Perceptron update rule:

$$w = w + \eta\, y\, x$$

where:

- w= weight vector
- η= learning rate
- y= true class label ($+1$ or -1)
- x= input feature vector
- Update the **bias** along with the weights.
- Choose a suitable learning rate (e.g., **0.01** or **0.1**).
- Encode class labels consistently as **+1** and **−1**.
- Normalize or standardize input features.
- Shuffle the training data before each epoch.
- Verify that the dataset is truly linearly separable and free from labeling errors.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

a) Identify possible mathematical or coding errors in gradient computation.

Answer:

Possible errors include:

- Incorrect derivative of the activation function (e.g., wrong derivative of ReLU or Sigmoid).
- Wrong application of the chain rule during backpropagation.
- Incorrect sign in gradient computation, causing gradient ascent instead of gradient descent.
- Errors in matrix multiplication or tensor dimensions.
- Incorrect weight or bias update equation.
- Forgetting to average gradients over the batch.
- Learning rate set too high or too low.
- Gradients not reset before each update, causing accumulation from previous iterations.

---

b) Explain the role of activation functions in gradient flow.

Answer:

Activation functions determine how gradients are propagated through the network during backpropagation.

- **Sigmoid:** Can cause very small gradients for large positive or negative inputs, leading to slow learning.
- **Tanh:** Has stronger gradients than Sigmoid near zero but can still suffer from vanishing gradients.
- **ReLU (Rectified Linear Unit):** Allows gradients to pass for positive inputs, reducing the vanishing gradient problem. However, neurons receiving only negative inputs may stop learning ("dead ReLU").
- **Leaky ReLU/ELU:** Allow small gradients even for negative inputs, reducing the dead neuron problem.

Choosing an appropriate activation function helps maintain stable gradient flow and improves learning efficiency.

---

c) Analyze how vanishing or exploding gradients affect training.

Answer:

Vanishing Gradients:

- Gradients become extremely small as they propagate backward.
- Early layers receive almost no updates.
- Training becomes very slow or stops completely.
- Common with deep networks using Sigmoid or Tanh activations.

Exploding Gradients:

- Gradients become extremely large during backpropagation.
- Weight updates become unstable.

- Loss increases instead of decreasing.
- Training may diverge or produce NaN (Not a Number) values.

Both problems prevent the neural network from learning effectively.

---

d) Suggest techniques to fix the issue (e.g., normalization, initialization).

Answer:

To improve training stability and ensure proper convergence:

- Use **ReLU**, **Leaky ReLU**, or **ELU** instead of Sigmoid in deep networks.
- Apply **Xavier (Glorot) Initialization** for Sigmoid/Tanh networks.
- Apply **He Initialization** for ReLU-based networks.
- Normalize inputs using **feature normalization** or **standardization**.
- Use **Batch Normalization** to stabilize activations and gradient flow.
- Apply **Gradient Clipping** to prevent exploding gradients.
- Reduce the learning rate if the loss is increasing.
- Use adaptive optimizers such as **Adam** or **RMSprop**.
- Verify the backpropagation equations and weight update implementation.

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.
Tasks:

a) Explain why Batch Gradient Descent is slow in large datasets.

Answer:

Batch Gradient Descent (BGD) computes the gradient using the **entire training dataset** before updating the model parameters.

As a result:

- Every update requires processing all training samples.
- Computation time per iteration is very high for large datasets.
- Memory usage is high because the entire dataset must be accessed.
- The model updates weights only once per epoch, making learning slower.
- Training takes longer even though the loss decreases smoothly.

---

b) Analyze the cause of fluctuations in SGD.

Answer:

Stochastic Gradient Descent (SGD) updates the model parameters **after each training sample**.

This causes fluctuations because:

- Each sample provides a noisy estimate of the true gradient.
- Different samples may point the optimization in different directions.
- The loss may increase temporarily before decreasing again.
- Frequent updates create a zigzag optimization path.
- Although noisy, these fluctuations can help the model escape local minima and saddle points.

---

c) Compare the error behavior of Mini-Batch Gradient Descent.

Answer:

Mini-Batch Gradient Descent uses a **small batch of samples** (e.g., 32, 64, or 128) for each update.

Error behavior:

- Less noisy than SGD because gradients are averaged over multiple samples.
- Faster than Batch Gradient Descent since it does not process the entire dataset before each update.
- Produces smoother loss curves than SGD.
- Requires less memory than Batch Gradient Descent.
- Provides a good balance between training speed and convergence stability.

---

d) Recommend the best approach and justify with error trade-offs.

Answer:

Mini-Batch Gradient Descent is generally the best approach.

Justification:

- It combines the advantages of both Batch Gradient Descent and SGD.
- Converges faster than Batch Gradient Descent.
- Has smaller loss fluctuations than SGD.
- Efficiently utilizes modern GPU and CPU hardware.
- Produces stable parameter updates while maintaining good computational efficiency.
- It is the most commonly used optimization method in deep learning.

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

a) Explain how high dimensionality increases model error.

Answer:

High dimensionality increases model error because:

- The number of features becomes much larger than the number of training samples.
- The model learns **noise** instead of the actual patterns in the data.
- Data becomes sparse, making it difficult to identify meaningful relationships.
- The model becomes overly complex, resulting in poor generalization.
- Training accuracy becomes high, but test accuracy decreases due to overfitting.

---

b) Identify signs of overfitting in this scenario.

Answer:

Signs of overfitting include:

- Very high training accuracy but low test accuracy.
- Low training error and high testing error.
- The model performs well on known data but poorly on unseen data.
- Large gap between training and validation performance.
- High variance in model predictions.

---

c) Analyze why distance-based learning fails in high dimensions.

Answer:

Distance-based algorithms (such as **K-Nearest Neighbors (KNN)** and **K-Means**) rely on measuring distances between data points.

In high-dimensional spaces:

- Data points become very sparse.
- Distances between the nearest and farthest points become nearly the same.
- It becomes difficult to identify true nearest neighbors.

- Similarity measures lose their effectiveness.
- As a result, classification and clustering accuracy decreases.

---

d) Suggest dimensionality reduction or regularization techniques to improve performance.

Answer:

To improve model performance:

Dimensionality Reduction Techniques:

- **Principal Component Analysis (PCA)** – Reduces the number of features while preserving most of the data variance.
- **Linear Discriminant Analysis (LDA)** – Reduces dimensions while maximizing class separation.
- **Autoencoders** – Learn compact feature representations using neural networks.
- **Feature Selection** – Remove irrelevant or redundant features.

Regularization Techniques:

- **L1 Regularization (Lasso)** – Removes unnecessary features by shrinking some weights to zero.
- **L2 Regularization (Ridge)** – Prevents very large weights and reduces model complexity.
- **Dropout** (for neural networks) – Randomly deactivates neurons during training to reduce overfitting.
- **Early Stopping** – Stops training before the model starts memorizing the training data.

These techniques reduce overfitting and improve the model's ability to generalize to unseen data.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, wellstructured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, wellorganized, and properly explained	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand
		solution			